

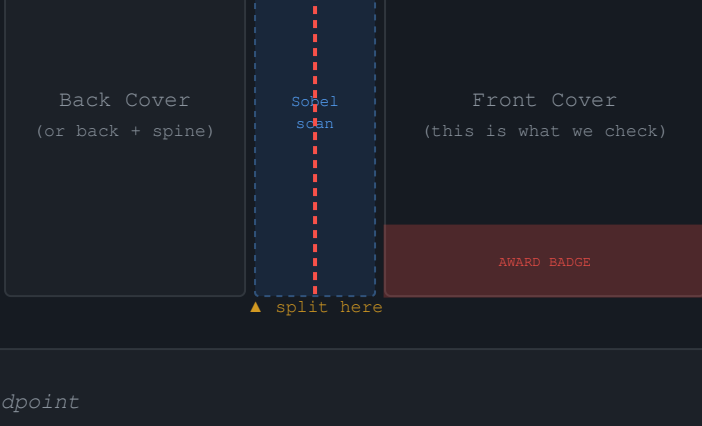
Cover Validation Algorithm

A visual walkthrough of how the system detects text overlap with the award badge zone on book covers.

1 Spread Split → 2 Zone Compute → 3 OCR Pipeline → 4 Fuzzy Match → 5 Verdict

STEP 1 Spread Splitting

Covers often arrive as wraparound spreads — back cover + spine + front cover in one image. The algorithm finds where to split them.



```
# Compute Sobel edge energy near the midpoint
mag = sobel(gray_image)

# The column with the highest edge energy = gutter fold
split_col = argmax(mag[:, mid - 40 : mid + 40])

# If no strong edge found, just split 50/50
back_cover, front_cover = image[:, :split_col], image[:, split_col:]
```

STEP 2 Zone Computation

From the front cover dimensions, we compute three critical zones in real-world millimeters, then convert to pixels.

5" × 8"

Standard trim size

3 mm

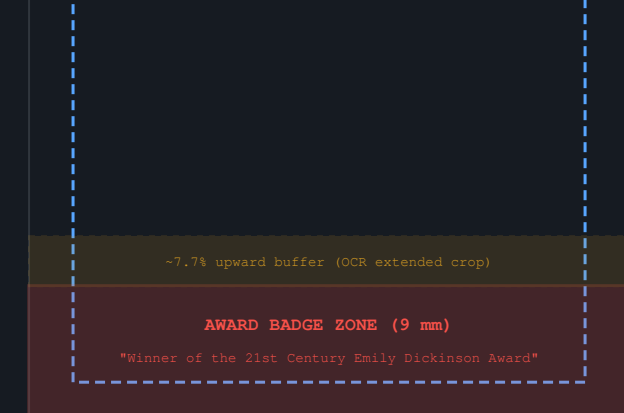
Safe margin (all edges)

9 mm

Badge zone height (bottom)

1000 px

Minimum front-cover width



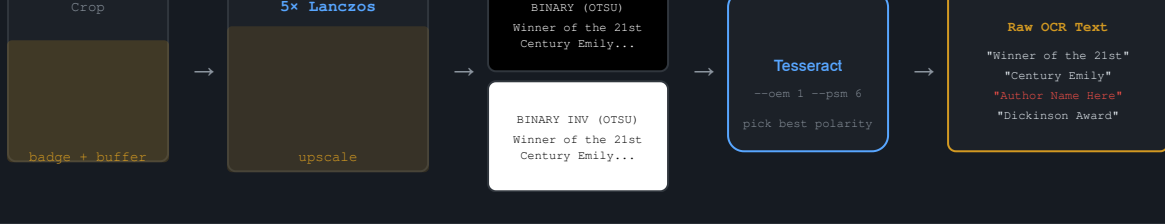
```
# Estimate DPI from pixel dimensions and physical trim size
dpi = (front_w / 5.0 + front_h / 8.0) / 2.0
px_per_mm = dpi / 25.4

# Badge zone = bottom 9 mm of the front cover
badge_zone = Rect(x=0, y=front_h - round(9 * px_per_mm),
                  w=front_w, h=round(9 * px_per_mm))

# Safe area = 3 mm inward from each edge
safe_area = Rect(x=side_px, y=side_px,
                  w=front_w - 2*side_px, h=front_h - side_px - bottom_px)
```

STEP 3 OCR Pipeline

The badge zone (plus upward buffer) is cropped, upscaled 5×, binarized with Otsu in both polarities, and fed to Tesseract.



```
# Upcrop + 5x upscale + Otsu binarization (both polarities)
upscaled = cv2.resize(badge_crop, None, fx=5, fy=5,
                      interpolation=cv2.INTER_LANCZOS4)
_, bin_norm = cv2.threshold(upscaled, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
_, bin_inv = cv2.threshold(upscaled, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Run both and keep the one with more alphabetic content
text_norm = pytesseract.image_to_string(bin_norm, config="--oem 1 --psm 6")
text_inv = pytesseract.image_to_string(bin_inv, config="--oem 1 --psm 6")
result = text_inv if alpha_count(text_inv) > alpha_count(text_norm) else text_norm
```

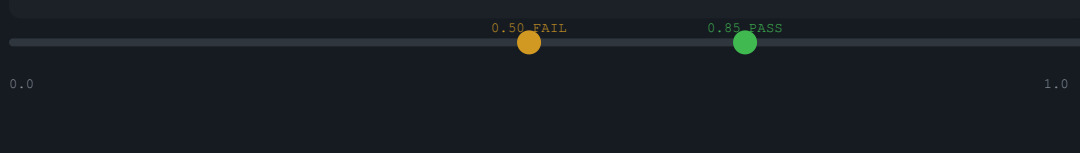
STEP 4 Fuzzy Matching & Leftover Extraction

The raw OCR text is fuzzy-matched against the known badge phrase. Lines that don't match become "leftovers" — candidates for text overlap.

```
BADGE_PHRASE = "Winner of the 21st Century Emily Dickinson Award"

# Fuzzy match the entire OCR output against the badge phrase
match_ratio = SequenceMatcher(None,
                               normalize(raw_text),
                               normalize(BADGE_PHRASE)).ratio()

# For each line: does it look like the badge? Discard if yes.
for line in raw_text_lines:
    if match_against_badge(line) >= 0.85:
        continue # this is the badge itself
    leftovers.append(line) # possible intruding text!
```



Matching tiers:

- match ≥ 0.85** — Badge is fully legible. Leftovers are filtered permissively (allows near-miss badge lines).
- 0.50 ≤ match < 0.85** — Badge partially readable. Leftovers are run through a strict "clean readability" filter (alphabetic density ≥ 70%, ≥ 2 words of length ≥ 3, avg word length > 2.5).
- match < 0.50** — Badge garbled. A relaxed check looks for any readable non-badge sentence (alphabetic density > 40%, word-like chunks > 6 chars).

STEP 5 Three-Tier Verdict

The final decision combines OCR leftovers, match ratio, MSER geometric overlap count, and a resolution guard.

✓ PASS

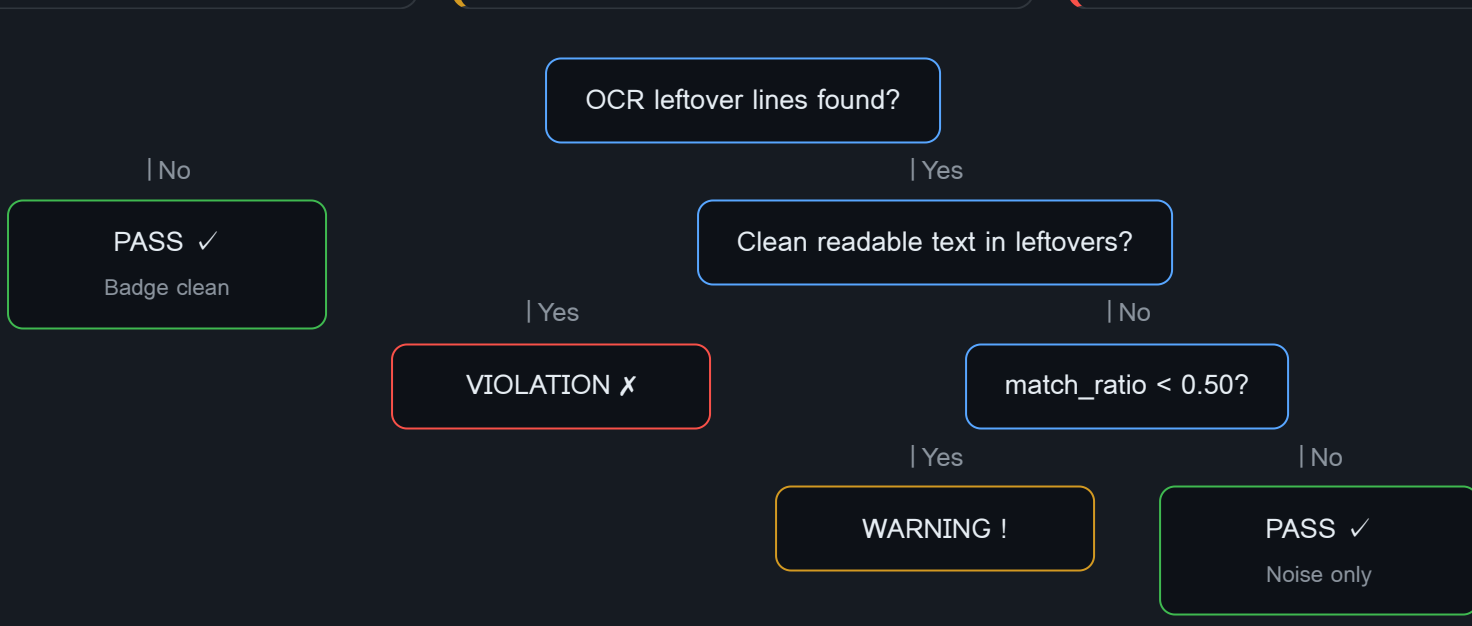
Badge is fully legible (match ≥ 0.85) with no intruding text. Or mid-range match with only OCR noise — no overlap concerns.

! WARNING

Leftover OCR lines exist but none pass the clean-readability filter. Match ratio < 0.50 suggests edge proximity or layout congestion, not confirmed overlap.

✗ VIOLATION

Clean, readable non-badge text lines are confirmed in the badge zone. The cover must be redesigned with adequate clearance.



```
if clean_leftovers:
    return "VIOLATION" # readable non-badge text in badge zone

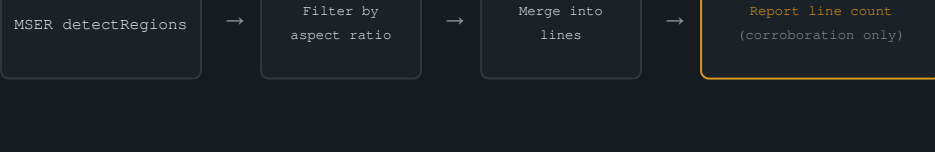
if not leftovers:
    return "PASS" # badge zone clean

if match_ratio < 0.50:
    return "WARNING" # edge proximity, no confirmed overlap

return "PASS" # mid-range match, OCR noise only
```

+ MSER Geometric Text Detection (Secondary)

A corroborating signal using Maximally Stable Extremal Regions to detect text-like blobs in the badge zone. Never the sole basis for a VIOLATION.



- Uses **MSER** (OpenCV) to detect extremal regions — blobs that are either consistently darker or lighter than their surroundings.
- Glyphs filtered by aspect ratio: > 0.15 and < 4.0 (discards lines and blobs that are too tall or wide to be text).
- Merges nearby glyphs into lines using y-overlap and x-distance thresholds.
- Filters out merged lines smaller than 40 px wide or 8 px tall.
- The **count** of geometric lines in the badge zone is reported alongside the OCR verdict for informational purposes only.

⚡ Resolution Guard

Covers narrower than 1000 px on the front face get flagged. The detection still runs but results are less reliable.

1000 px

Minimum width

~12 px

Badge zone at 1000 px width

< 12 px

Too few pixels for reliable OCR

```
if front_w < 1000:
    resolution_warning = (
        "Front cover is only {front_w}px wide "
        "(min 1000 px recommended). "
        "Too small for precise measurement."
    )
    confidence_score -= 10 # penalize confidence
```

The combined confidence score starts at 90 (high), 70 (medium), or 50 (low) and is further adjusted by the match ratio and resolution penalty — clamped to [10, 99].

🖨 Visual Output Legend

For WARNING and VIOLATION cases, the system generates an annotated PNG that makes the problem immediately obvious.

✗ VIOLATION [high] match=0.62 Front cover is only 750px wide (min 1000 px recommended).

Badge zone (red = VIOLATION) Safe margin (cyan) Upward buffer (amber) OCR sidebar + intrusion list

📄 Ground Truth Results

The test set has 8 labelled covers. Run `python cover_verify.py` to check all against expected verdicts.

#	Title	Expected	Notes
28	Shabd (uncorrected)	VIOLATION	Author name overlaps badge text
29	Shabd (corrected)	PASS	Clean separation after fix
31	Offline Sorrows / Online Ghosts	PASS	Human review cleared
32	Echoes Along the Way (uncorrected)	VIOLATION	Tagline overlaps badge
33	Echoes Along the Way (corrected)	PASS	Clean separation after fix
34	Inner Mirror (uncorrected)	WARNING	Content near badge; technically allowed but tight
35	Inner Mirror (corrected)	PASS	Breathing room added
36	Tainted By Emotion	VIOLATION	Tagline overlaps; floral border crowds margins

📁 Key Source Files

File	What it does
<code>src/detector.py</code>	Core algorithm: <code>check_badge_overlap()</code> , OCR, MSER, zones, verdict
<code>src/visualizer.py</code>	Annotated PNG generation with <code>all_overlay</code> elements
<code>src/models.py</code>	Pydantic response model + <code>build_response()</code> factory
<code>src/main.py</code>	FastAPI server, CORS, static mounts, startup
<code>src/routers/verify.py</code>	POST <code>/verify</code> endpoint — upload, process, respond
<code>src/database.py</code>	SQLite revision tracking (books + revisions tables)
<code>src/pdf_converter.py</code>	PDF → numpy array via PyMuPDF
<code>src/filename_parser.py</code>	ISBN extraction from <code>ISBN_text.txt</code> file names
<code>cover_verify.py</code>	Standalone batch script with ground-truth comparison
<code>cover_visualize.py</code>	Standalone batch script + annotated visual output